

PHASE 21 — COMPLETE SYSTEM ARCHITECTURE, TECH STACK, DATABASE SCHEMA, APIs & IMPLEMENTATION BLUEPRINT

Project: Enterprise Cryptographic Discovery & Analysis Tool (ECDAT)

Problem Statement: SIH26164

Organization: National Technical Research Organisation (NTRO)

Theme: Blockchain & Cybersecurity

Phase: 21 of 22

Status: Full Technical Architecture, Data Model, API Design, Repository Structure & Implementation Blueprint

1. Purpose of This Phase

The previous phases defined **what ECDAT should become**.

This phase defines:

Exactly how we should build it.

The objective is to convert the conceptual architecture into a concrete technical system that your team can divide among developers and implement.

The system must connect:

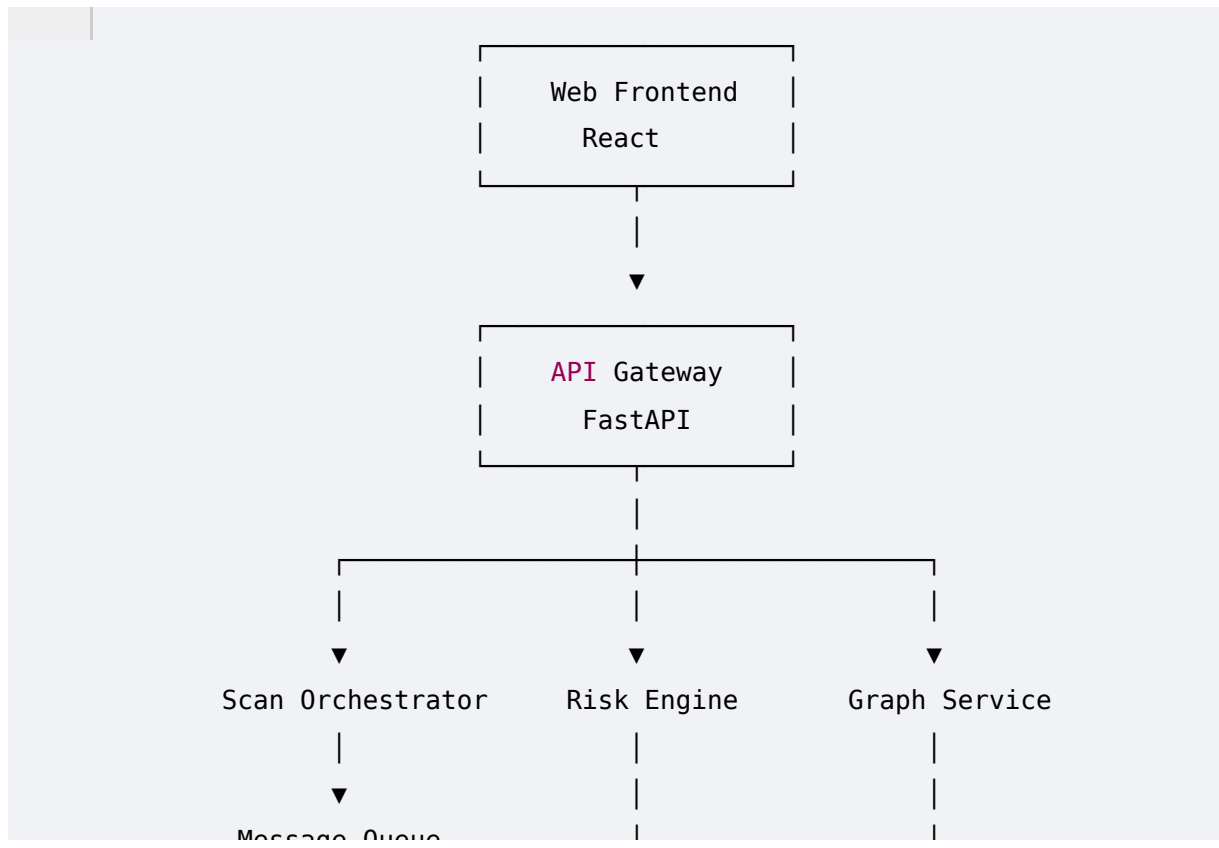
```
Repository
  ↓
Discovery
  ↓
Crypto Detection
  ↓
Evidence
  ↓
CBOM
  ↓
Knowledge Graph
  ↓
Risk Engine
  ↓
PQC Recommendation
```

↓
Migration Planner

2. The Recommended Architecture

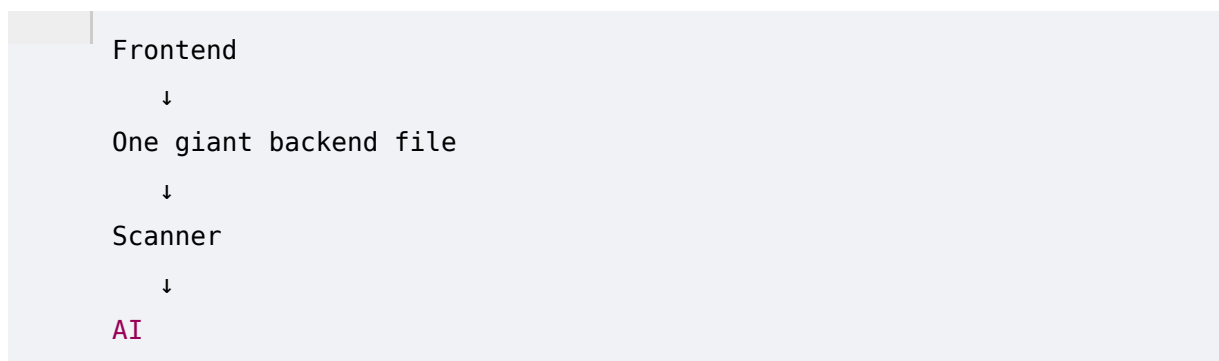
ECDAT should use a modular architecture rather than an enormous monolithic application.

Recommended model:



3. Architectural Principle

Do not build:



Instead:

```
Independent modules
+
Clear interfaces
+
Typed data contracts
+
Event-driven scanning
```

This will make the system much easier to extend.

4. Recommended Technology Stack

Frontend

```
React
TypeScript
Next.js
Tailwind CSS
React Query
ECharts / Recharts
React Flow
```

Backend

```
Python
FastAPI
Pydantic
SQLAlchemy
Alembic
```

Python is particularly useful because the project is heavily dependent on:

```
AST analysis
Binary analysis
Security tooling
AI/ML
Data processing
```

5. Scanner Technology

Recommended:

- Python
- Tree-sitter
- Semgrep-style rules
- YARA
- LIEF
- OpenSSL tooling
- Trivy
- Syft
- Custom parsers

Not every tool needs to be used in the first prototype.

6. Database

Primary relational database:

- PostgreSQL

Store:

- Users
- Projects
- Applications
- Assets
- Findings
- CBOM
- Policies
- Risk
- Migration
- Audit

7. Graph Database

For the prototype:

- Neo4j

or PostgreSQL + graph extensions if simplifying deployment.

For the SIH prototype, Neo4j provides an excellent visualization story.

8. Object Storage

Use:

S3-compatible storage

Possible implementations:

MinIO

AWS S3

Azure Blob

GCP Cloud Storage

For a local prototype:

MinIO

is ideal.

9. Queue

Recommended:

Redis

with:

Celery

or a lightweight task queue.

Architecture:

API

↓

Queue

↓

Worker

10. Cache

Redis can also provide:

Caching

Rate limiting

Temporary state

Job status
Distributed locks

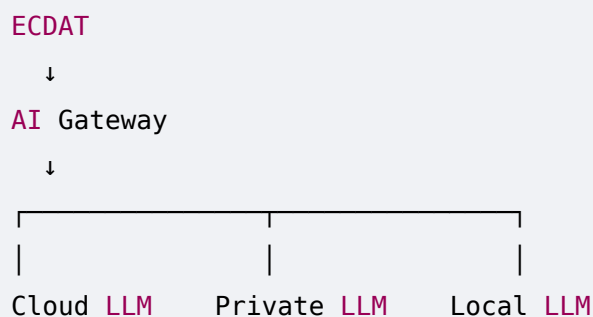
11. AI Layer

Do not tightly couple ECDAT to a single model.

Create:

AI Gateway

between ECDAT and models.



12. AI Gateway

Example abstraction:

```
class AIProvider:
    def generate(self, context, prompt):
        ...
```

Providers can include:

- OpenAI-compatible endpoint
- Local model
- Enterprise model
- Mock model

13. Why This Matters

If the AI provider changes:

Model A

to:

the rest of ECDAT should not require major changes.

14. Repository Structure

Recommended:

```
ecdat/  
|  
├─ frontend/  
|  
├─ backend/  
|   ├─ api/  
|   ├─ core/  
|   ├─ models/  
|   ├─ schemas/  
|   ├─ services/  
|   ├─ workers/  
|   ├─ scanners/  
|   ├─ risk/  
|   ├─ graph/  
|   ├─ ai/  
|   ├─ migration/  
|   └─ reports/  
|   └─ tests/
```

15. Backend Structure

```
backend/  
|  
├─ main.py  
|  
├─ api/  
|   ├─ auth.py  
|   ├─ scans.py  
|   ├─ assets.py  
|   ├─ findings.py  
|   ├─ cbom.py  
|   ├─ risk.py  
|   ├─ graph.py  
|   └─ recommendations.py
```

```
|   ├── migration.py
|   ├── reports.py
|   └── dashboard.py
|
```

16. Frontend Structure

```
frontend/
|
├── app/
|
├── components/
|   ├── dashboard/
|   ├── scans/
|   ├── cbom/
|   ├── graph/
|   ├── risk/
|   ├── migration/
|   └── ai/
|
├── hooks/
├── services/
├── types/
├── lib/
└── styles/
```

17. Core Domain Objects

ECDAT revolves around:

```
Organization
Project
Application
Repository
Scan
Finding
Crypto Asset
Evidence
CBOM
Dependency
Certificate
Key Metadata
```


Risk
Recommendation
Migration Plan
Policy
User

18. Organization

Organization

- |
- ├ Users
- ├ Projects
- ├ Policies
- └ Applications

19. Project

A project can represent:

Business Unit
Application Portfolio
Migration Program
Repository Group

20. Application

Example:

Application:
Payment Gateway

Owner:
Payments Team

Criticality:
Critical

Environment:
Production

21. Repository

```
Repository
├── URL
├── Branch
├── Commit
├── Language
└── Owner
```

22. Scan

Every scan should have:

```
Scan ID
Repository
Scanner Version
Ruleset Version
Commit
Started At
Completed At
Status
```

23. Scan States

```
QUEUED
RUNNING
COMPLETED
FAILED
CANCELLED
```

24. Finding

A finding represents a detected cryptographic artifact.

Example:

```
Finding:
RSA API usage

File:
src/security.py
```

Line:

42

Confidence:

0.97

25. Crypto Asset

Normalize findings into assets.

Example:

Crypto Asset:

RSA

Type:

Algorithm

Version:

N/A

Usage:

Signature

Location:

Payment API

26. Evidence

Every asset should have evidence.

Evidence

└─ Source

└─ File

└─ Line

└─ Snippet Hash

└─ Scan

27. CBOM Record

Conceptual:

```
{
  "asset_id": "crypto-123",
  "type": "algorithm",
  "name": "RSA",
  "usage": "signature",
  "key_size": 2048,
  "confidence": 0.97,
  "source": "src/security.py",
  "line": 42
}
```

28. PostgreSQL Schema

A simplified database:

```
organizations
users
roles
projects
applications
repositories
scans
findings
crypto_assets
evidence
dependencies
certificates
risk_assessments
recommendations
migration_plans
policies
audit_events
```

29. Organizations

```
organizations
-----
id
name
created_at
```

30. Users

```
users
-----
id
organization_id
email
name
role_id
status
created_at
```

31. Applications

```
applications
-----
id
project_id
name
description
owner
criticality
environment
created_at
```

32. Repositories

```
repositories
-----
id
application_id
url
branch
commit_hash
language
created_at
```

33. Scans

```
scans
```

```
-----
```

```
id
```

```
repository_id
```

```
status
```

```
scanner_version
```

```
ruleset_version
```

```
started_at
```

```
completed_at
```

```
error
```

34. Findings

```
findings
```

```
-----
```

```
id
```

```
scan_id
```

```
asset_id
```

```
file_path
```

```
line_start
```

```
line_end
```

```
detector
```

```
confidence
```

```
severity
```

```
created_at
```

35. Crypto Assets

```
crypto_assets
```

```
-----
```

```
id
```

```
name
```

```
type
```

```
family
```

```
version
```

```
usage
```

```
key_size
```

```
status
```

36. Evidence

```
evidence
-----
id
finding_id
source_type
source_reference
content_hash
snippet
created_at
```

For sensitive environments, consider storing only a minimized or redacted snippet.

37. Risk Assessment

```
risk_assessments
-----
id
asset_id
quantum_risk
business_criticality
data_lifetime
migration_time
exposure
mosca_score
overall_score
created_at
```

38. Recommendation

```
recommendations
-----
id
asset_id
target_algorithm
strategy
reason
confidence
```

```
policy_version
created_at
```

39. Migration Plan

```
migration_plans
-----
id
asset_id
current_state
target_state
effort
status
owner
created_at
```

40. Migration Tasks

```
migration_tasks
-----
id
migration_plan_id
title
description
owner
status
priority
dependency_id
```

41. Audit Events

```
audit_events
-----
id
organization_id
actor_id
event_type
resource_type
resource_id
```



```
metadata
timestamp
```

42. Graph Model

Neo4j model:

```
(:Application)
(:Repository)
(:Library)
(:Algorithm)
(:Certificate)
(:Key)
(:Protocol)
(😊ataset)
(:BusinessProcess)
(:Risk)
(:Migration)
```

43. Graph Relationships

Examples:

```
Application -USES→ Library

Library -IMPLEMENTS→ Algorithm

Application -USES→ Algorithm

Certificate -PROTECTS→ Application

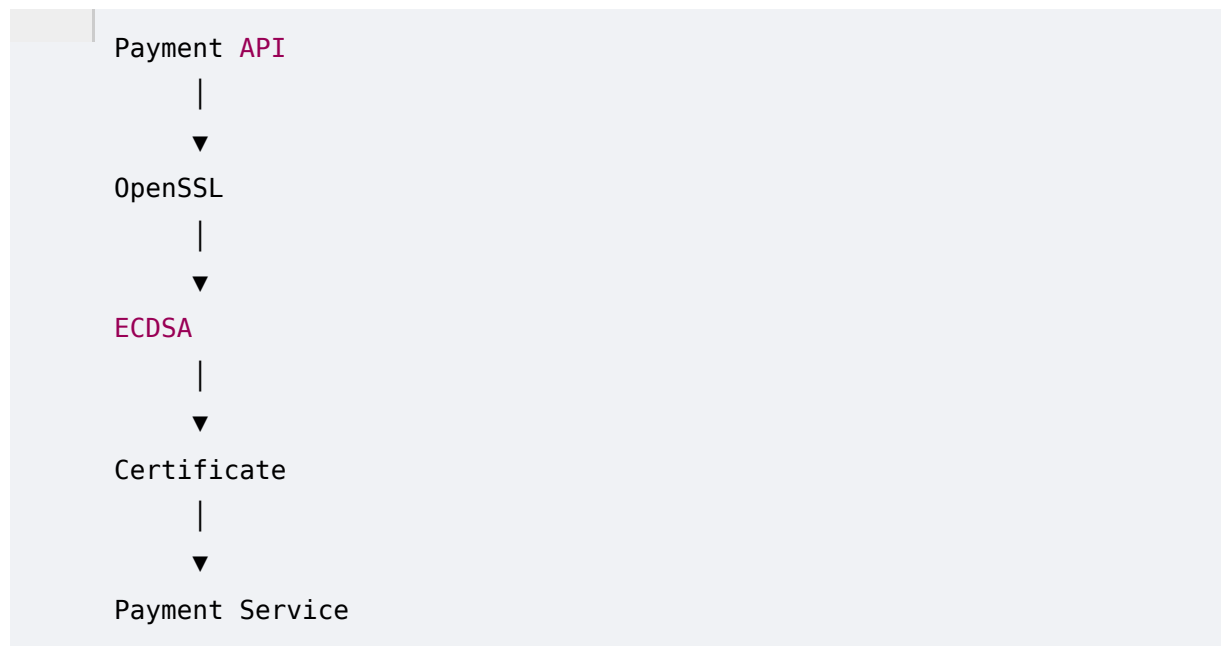
Application -PROCESSES→ Dataset

Dataset -HAS_LIFETIME→ DataLifetime

Algorithm -HAS_RISK→ Risk
```

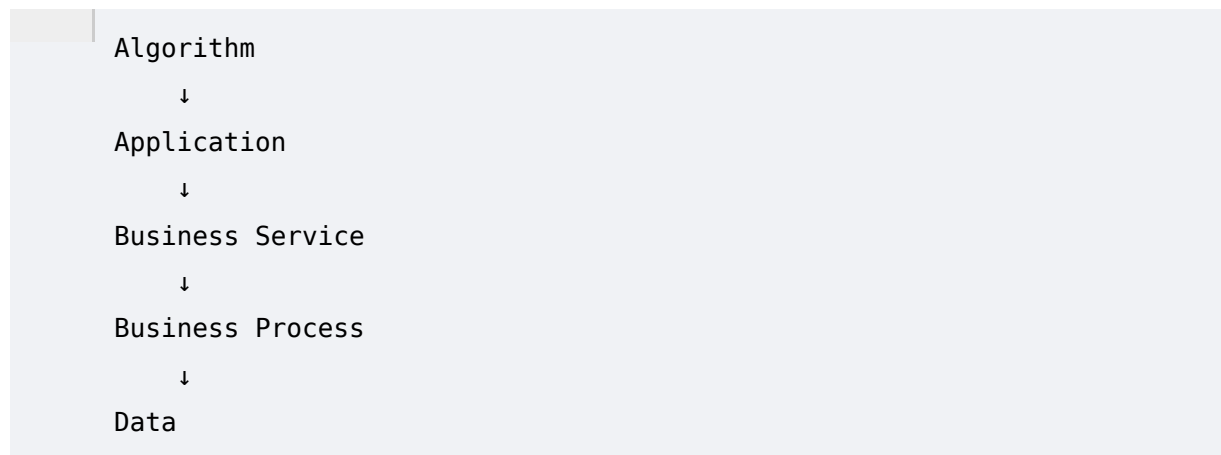
44. Dependency Graph

Example:



45. Business Graph

Connect technical and business layers:



This enables business impact analysis.

46. Graph Query Example

Question:

Which critical applications depend on RSA?

Conceptually:

```
MATCH (a:Application)-[*]->(alg:Algorithm)
WHERE alg.name = "RSA"
AND a.criticality = "critical"
RETURN a
```

47. Blast Radius Query

Question:

What is affected if OpenSSL is compromised?

Graph traversal:

```
OpenSSL
  ↓
Libraries
  ↓
Applications
  ↓
Business Processes
```

48. API Architecture

Use REST initially.

Base:

```
/api/v1
```

49. Authentication API

```
POST /auth/login
POST /auth/refresh
POST /auth/logout
GET  /auth/me
```

For enterprise deployment, federate authentication to the organization's identity provider rather than building a custom password system unnecessarily.

50. Scan API

```
POST /scans
GET  /scans
GET  /scans/{id}
POST /scans/{id}/cancel
```

51. Scan Request

```
{
  "repository_id": "repo-123",
  "branch": "main",
  "scan_type": "full"
}
```

52. Scan Response

```
{
  "scan_id": "scan-123",
  "status": "queued"
}
```

53. Finding API

```
GET /findings
GET /findings/{id}
PATCH /findings/{id}
```

54. CBOM API

```
GET /cbom
GET /cbom/{asset_id}
GET /cbom/export
```

Export formats:

```
JSON
CSV
CycloneDX-compatible representation where applicable
```

55. Risk API

```
GET /risks
GET /risks/{asset_id}
GET /risks/summary
```

56. Graph API

```
GET /graph/application/{id}
GET /graph/asset/{id}
GET /graph/blast-radius/{id}
```

57. Recommendation API

```
GET /recommendations
GET /recommendations/{id}
POST /recommendations/{id}/review
```

58. Migration API

```
POST /migration/plans
GET /migration/plans
GET /migration/plans/{id}
PATCH /migration/plans/{id}
POST /migration/plans/{id}/verify
```

59. Report API

```
POST /reports
GET /reports/{id}
GET /reports/{id}/download
```

60. Dashboard API

```
GET /dashboard/summary
GET /dashboard/risk
GET /dashboard/algorithms
GET /dashboard/migration
```

61. Event Architecture

Long-running scans should not rely on HTTP requests staying open.

Use:

API



Queue



Worker



Events

62. Scan Event Flow

SCAN_CREATED



SCAN_STARTED



DISCOVERY_STARTED



DISCOVERY_COMPLETED



CBOM_GENERATED



RISK_CALCULATED



GRAPH_UPDATED



SCAN_COMPLETED

63. WebSocket / SSE

The UI can receive:

Scan :

43%

then:

Scan :

78%

then:

Completed

Server-Sent Events are sufficient for many prototype use cases.

64. Scanner Interface

Every scanner should implement a common interface.

```
class Scanner:
    def scan(self, target):
        ...
```

Output:

```
Finding[]
```

65. Source Scanner

```
Source
↓
Language Detection
↓
Parser
↓
AST
↓
Crypto Rules
↓
Findings
```

66. Dependency Scanner

```
Package Manifest
↓
Dependency Resolver
↓
Known Libraries
↓
Crypto Mapping
```

67. Certificate Scanner

Certificate

↓

Parser

↓

Algorithm

↓

Key Size

↓

Validity

↓

Chain

68. Binary Scanner

Binary

↓

File Type

↓

Strings

↓

Imports

↓

Symbols

↓

Crypto Signatures

69. Container Scanner

Image

↓

Manifest

↓

Layers

↓

Packages

↓

Binaries

↓

Configurations

70. Scanner Aggregator

All scanner outputs enter:

```
Finding Normalizer
```

71. Normalization

Different scanners might say:

```
RSA-2048
RSA 2048
RSA2048
```

Normalize:

```
algorithm = RSA
key_size = 2048
```

72. Entity Resolution

Resolve:

```
OpenSSL
openssl
OpenSSL library
libcrypto
```

into the correct canonical entity where evidence supports that relationship.

73. Canonical Algorithm Registry

Maintain:

```
algorithm_registry
```

with:

```
Canonical Name
Aliases
Family
Security Status
```

PQC Status
Approved Usage

74. Rule Engine

Rules should be externalized.

Example:

```
id: crypto.python.rsa
language: python
pattern: rsa.generate_private_key
algorithm: RSA
confidence: 0.95
```

75. Why External Rules?

Because security rules change faster than application architecture.

You should be able to add:

```
New detector
```

without rewriting the backend.

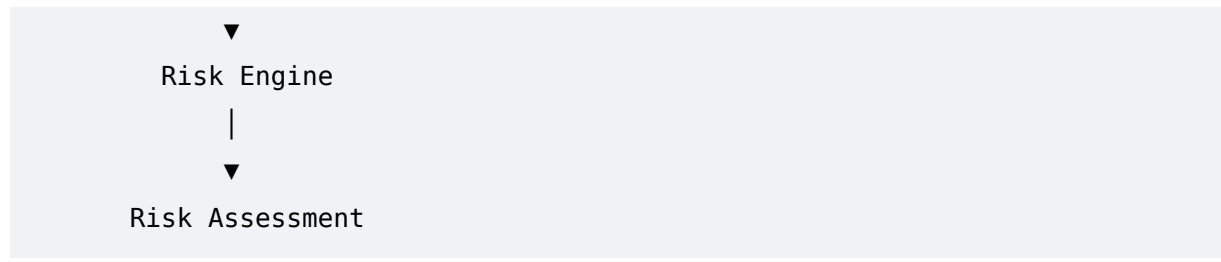
76. Rule Versioning

Every scan should record:

```
Ruleset:
v1.4.2
```

77. Risk Engine Architecture

```
CBOM
|
├─ Algorithm
├─ Key Size
├─ Usage
├─ Exposure
├─ Data Lifetime
└─ Business Criticality
    |
```



78. Risk Configuration

Avoid hardcoding every threshold.

Use:

```
risk:
  criticality_weight: 0.25
  data_lifetime_weight: 0.20
  algorithm_weight: 0.30
  exposure_weight: 0.15
  migration_weight: 0.10
```

79. Policy Engine

Policy decides:

```
Allowed
Warning
Forbidden
Migration Required
```

80. Example Policy

```
algorithm: SHA-1
usage: signature
status: forbidden
```

81. Recommendation Engine

Inputs:

```
Risk
Policy
Algorithm
Protocol
```

Library
Hardware
Performance
Compatibility

Output:

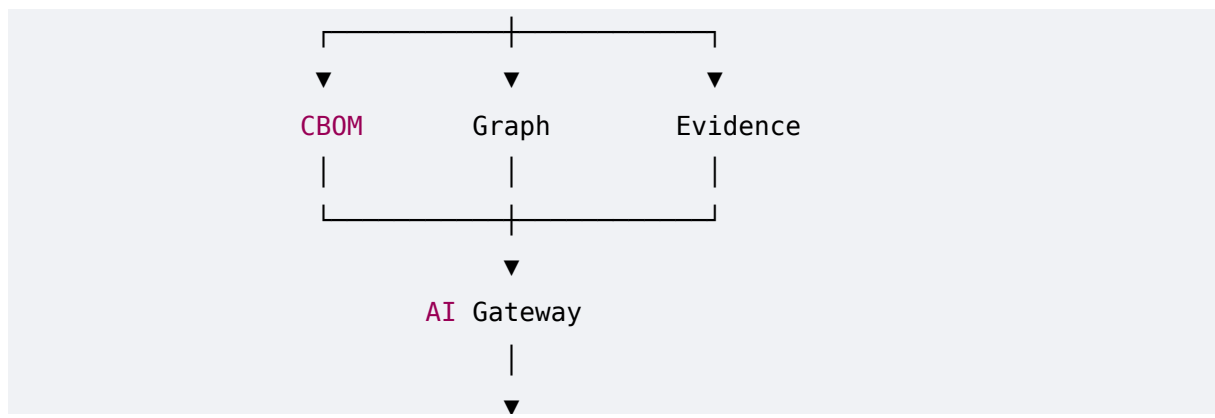
Target
Strategy
Reason
Confidence

82. Recommendation Pipeline

Current Crypto
↓
Eligibility
↓
Approved Alternatives
↓
Compatibility Filter
↓
Risk Filter
↓
Performance Filter
↓
Migration Cost
↓
Recommendation

83. AI RAG Architecture

User Question
|
▼
Query Interpreter
|
▼
Context Retrieval
|



84. Retrieval Sources

AI should retrieve from:

CBOM
Graph
Evidence
Policies
Algorithm Registry
Risk Engine
Migration Templates

85. AI Prompt Structure

Conceptually:

SYSTEM: You are ECDAT Security Analyst.
POLICY: Use only approved algorithms.
CONTEXT: [retrieved evidence]
QUESTION: [user request]

86. AI Output Schema

Do not accept arbitrary prose only.

Use structured output:

```
{  
  "answer": "...",  
  "findings": [],  
  "evidence": [],  
  "recommendations": [],  
  "confidence": 0.91  
}
```

87. Migration Engine

Inputs:

```
Current State  
Target State  
Dependencies  
Blockers  
Risk  
Organization Policy
```

Output:

```
Migration Plan
```

88. Migration Plan Structure

```
{  
  "asset_id": "crypto-123",  
  "current": "ECDSA",  
  "target": "approved-hybrid",  
  "steps": [],  
  "blockers": [],  
  "rollback": [],  
  "verification": []  
}
```

89. Report Engine

Reports should combine:

```
Executive Summary  
Inventory
```

- Risk
- Graph
- Recommendations
- Migration
- Evidence

90. Report Types

- Executive Report
- Technical Report
- Application Report
- Cryptographic Inventory
- Quantum Risk Report
- Migration Report
- Compliance Report

91. Executive Report

Focus on:

- Total assets
- Critical risks
- Business impact
- Migration progress
- Top recommendations

92. Technical Report

Focus on:

- Algorithms
- Libraries
- Certificates
- Findings
- Evidence
- Dependencies

93. Migration Report

Focus on:

```
Current
Target
Progress
Blockers
Owners
Timeline
```

94. Observability

Use:

```
Structured Logging
Metrics
Tracing
```

95. Metrics

Examples:

```
scans_total
scans_failed
scan_duration_seconds
findings_total
high_risk_assets
migration_progress
ai_requests_total
```

96. Tracing

Trace:

```
API request
↓
Scan job
↓
Worker
↓
Database
↓
Graph
```


This helps debug distributed failures.

97. Docker Architecture

For SIH:

```
docker-compose.yml
```

services:

```
frontend
backend
worker
postgres
neo4j
redis
minio
```

Optional:

```
ai-gateway
```

98. Docker Network

Separate:

```
frontend-network
backend-network
scanner-network
data-network
```

where practical.

99. Development Environment

One command should ideally launch:

```
docker compose up
```

and provide:

```
UI
API
```

```
Database
Graph
Worker
```

100. Environment Variables

Example:

```
DATABASE_URL
REDIS_URL
NEO4J_URI
S3_ENDPOINT
AI_PROVIDER
AI_MODEL
JWT_ISSUER
```

Secrets should not be committed.

101. Configuration Layers

```
Default
↓
Environment
↓
Enterprise Policy
↓
Runtime Configuration
```

102. Database Migrations

Use:

```
Alembic
```

Migration workflow:

```
Model change
↓
Migration
↓
Test
```

↓
Deploy

103. CI/CD Architecture

Git Push
↓
Lint
↓
Unit Tests
↓
Integration
↓
Ground Truth
↓
Security Scan
↓
Build
↓
SBOM
↓
Container Scan

104. Branching

A simple model:

main
develop
feature/*

For SIH, avoid excessive process overhead.

105. Pull Request Gates

Every PR should ideally pass:

Tests
Lint
Type Check
Security Scan

106. Infrastructure as Code

For production:

- Terraform
- Helm
- Kubernetes manifests

107. Deployment Profiles

Development

- Docker Compose
- Local **AI**
- Local databases

SIH

- Docker Compose
- Preloaded dataset
- Demo **AI**

Enterprise

- Kubernetes
- SSO**
- KMS**
- SIEM**
- Private **AI**

High Security

- Air-gapped
- Private **AI**
- HSM**
- Offline updates

108. MVP Architecture

Do NOT implement everything at once.

The MVP should contain:

- Source scanner
- Dependency scanner
- CBOM
- Risk engine
- Graph
- Dashboard
- Basic AI
- Basic recommendations

109. MVP Scanner Scope

Start with:

- Python
- Java
- JavaScript/TypeScript
- C/C++

and expand later.

110. MVP Crypto Scope

Start with:

- RSA
- ECDSA
- ECDH
- DH
- AES
- SHA-1
- SHA-2
- SHA-3
- HMAC
- TLS

Then add more.

111. MVP Certificate Scope

Support:

PEM
DER
X.509

with:

Algorithm
Key Size
Validity
Issuer
Subject

112. MVP Graph

Start with:

Application
Repository
Library
Algorithm
Certificate
Risk

Then expand.

113. MVP AI

Start with three queries:

Query 1

What cryptography is used?

Query 2

What are my highest quantum risks?

Query 3

What should I migrate first?

That is enough to demonstrate intelligence.

114. MVP Migration

Generate:

```
Current
Target
Reason
Steps
Blockers
```

Do not attempt automated production deployment.

115. MVP Verification

Simulate:

```
Before
↓
Migration
↓
After scan
```

and demonstrate risk reduction.

116. Team Architecture

A six-member team can divide roughly into:

```
Member 1:
Backend / Architecture

Member 2:
Source Scanner

Member 3:
CBOM / Risk

Member 4:
Graph / Data

Member 5:
Frontend
```

Everyone should still understand the whole system.

117. Development Order

Recommended:

1. Repository setup
2. Database
3. Scanner interface
4. Source scanner
5. CBOM
6. Risk engine
7. API
8. Dashboard
9. Graph
10. AI
11. Recommendation
12. Migration
13. Security
14. Testing
15. Demo

118. First Working Vertical Slice

Do not wait until every scanner is finished.

Build:

```
One repository
  ↓
One scanner
  ↓
One finding
  ↓
One CBOM asset
  ↓
One risk
  ↓
One dashboard card
```

Once that works:

Expand horizontally.

119. Vertical Slice Example

Repository:

demo-payment-api

Contains:

RSA signing

ECDAT:

```
Scan
  ↓
RSA detected
  ↓
CBOM
  ↓
High risk
  ↓
Dashboard
```

That becomes the foundation.

120. Second Vertical Slice

Add:

Certificate

Pipeline:

```
Certificate
  ↓
X.509
  ↓
RSA-2048
  ↓
Risk
```

↓
Graph

121. Third Vertical Slice

Add:

Library

Pipeline:

Application

↓

OpenSSL

↓

RSA

122. Fourth Vertical Slice

Add:

AI

Question:

Why is this risky?

123. Fifth Vertical Slice

Add:

Migration

Question:

What should we do?

124. Sixth Vertical Slice

Add:

Verification

Question:

125. Complete MVP Loop

```
graph TD; DISCOVER --> UNDERSTAND; UNDERSTAND --> ASSESS; ASSESS --> EXPLAIN; EXPLAIN --> RECOMMEND; RECOMMEND --> MIGRATE; MIGRATE --> VERIFY;
```

DISCOVER

↓

UNDERSTAND

↓

ASSESS

↓

EXPLAIN

↓

RECOMMEND

↓

MIGRATE

↓

VERIFY

126. Architecture Principle: Deterministic Core

This is extremely important.

The following should preferably be deterministic:

```
graph TD; Discovery; Normalization; CBOM; Risk; Policy; Graph; Migration_eligibility[Migration eligibility]; Verification;
```

Discovery

Normalization

CBOM

Risk

Policy

Graph

Migration eligibility

Verification

127. AI-Assisted Layer

AI should primarily handle:

```
graph TD; Explanation; Summarization; Natural_language_queries[Natural-language queries]; Developer_guidance[Developer guidance];
```

Explanation

Summarization

Natural-language queries

Developer guidance

Migration task drafting
Report generation

128. Why?

Because:

AI hallucination

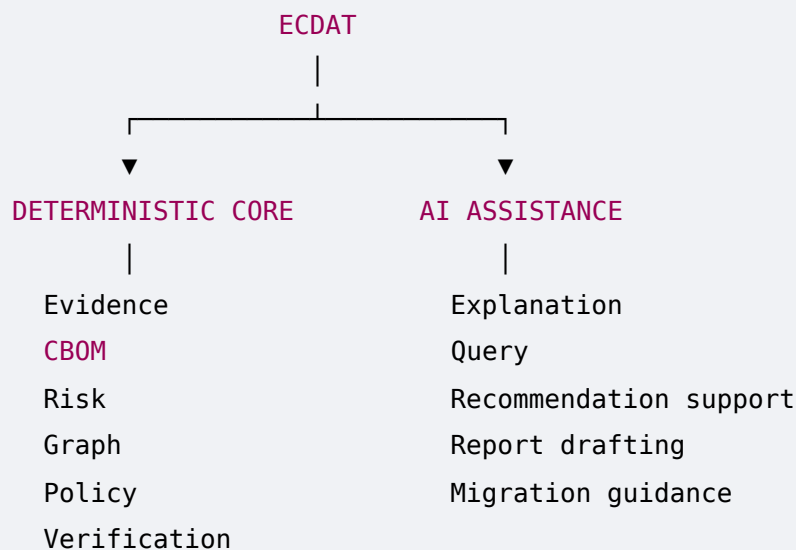
should never determine:

Security policy

or:

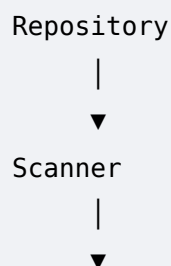
Actual asset existence

129. Architecture Separation



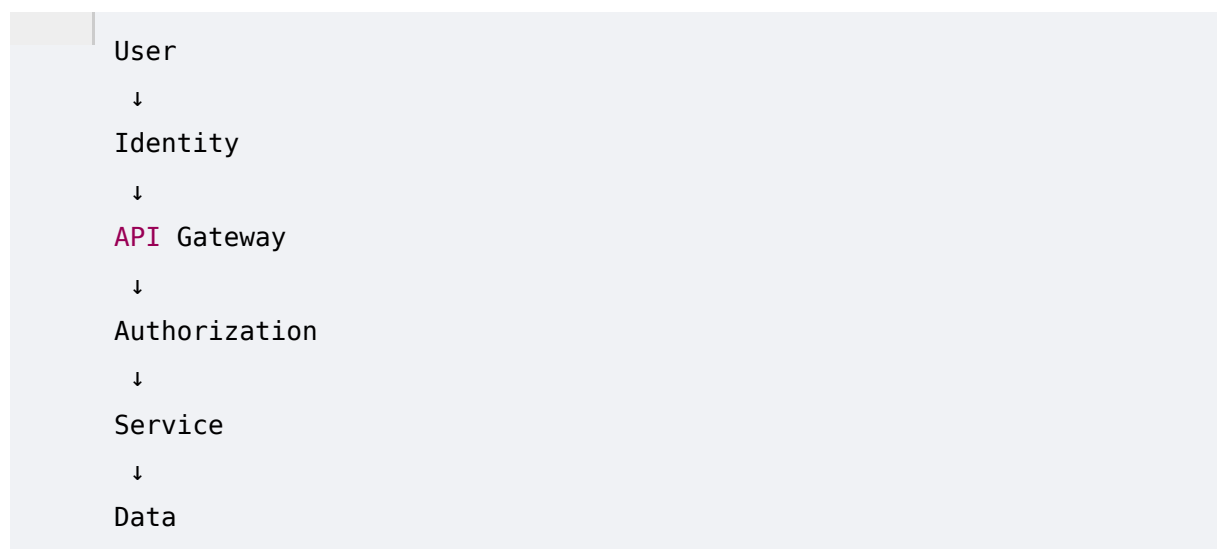
130. Data Flow

Complete flow:

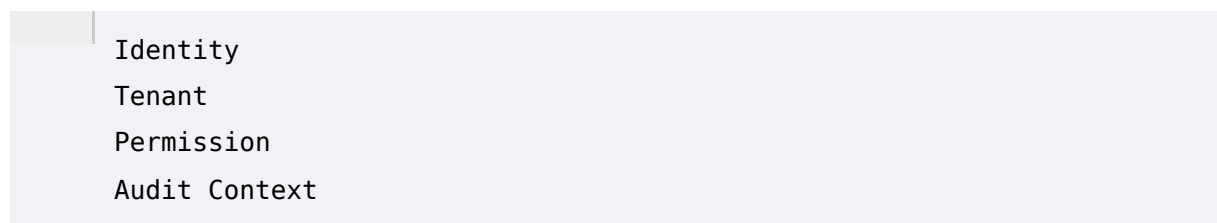




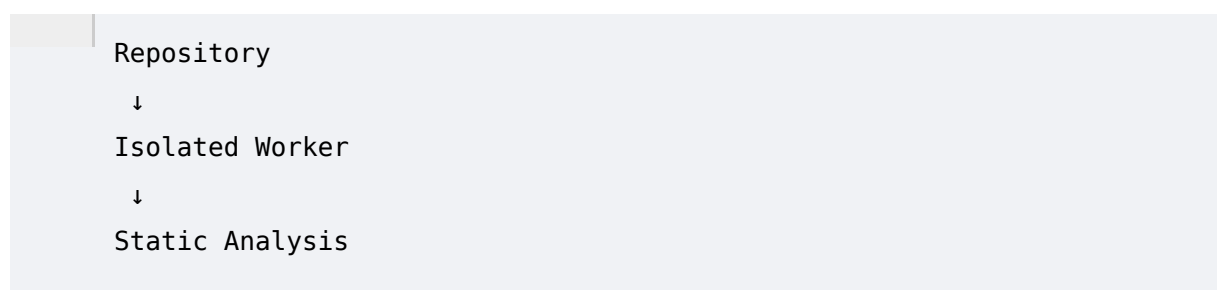
131. Security Data Flow



Every stage should preserve:



132. Scan Data Flow



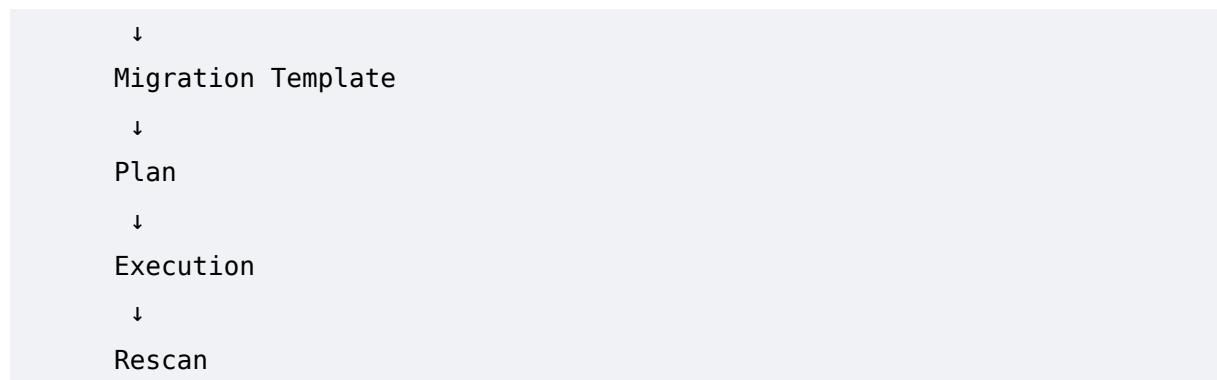
↓
Findings
↓
Sanitization
↓
CBOM
↓
Evidence

133. AI Data Flow

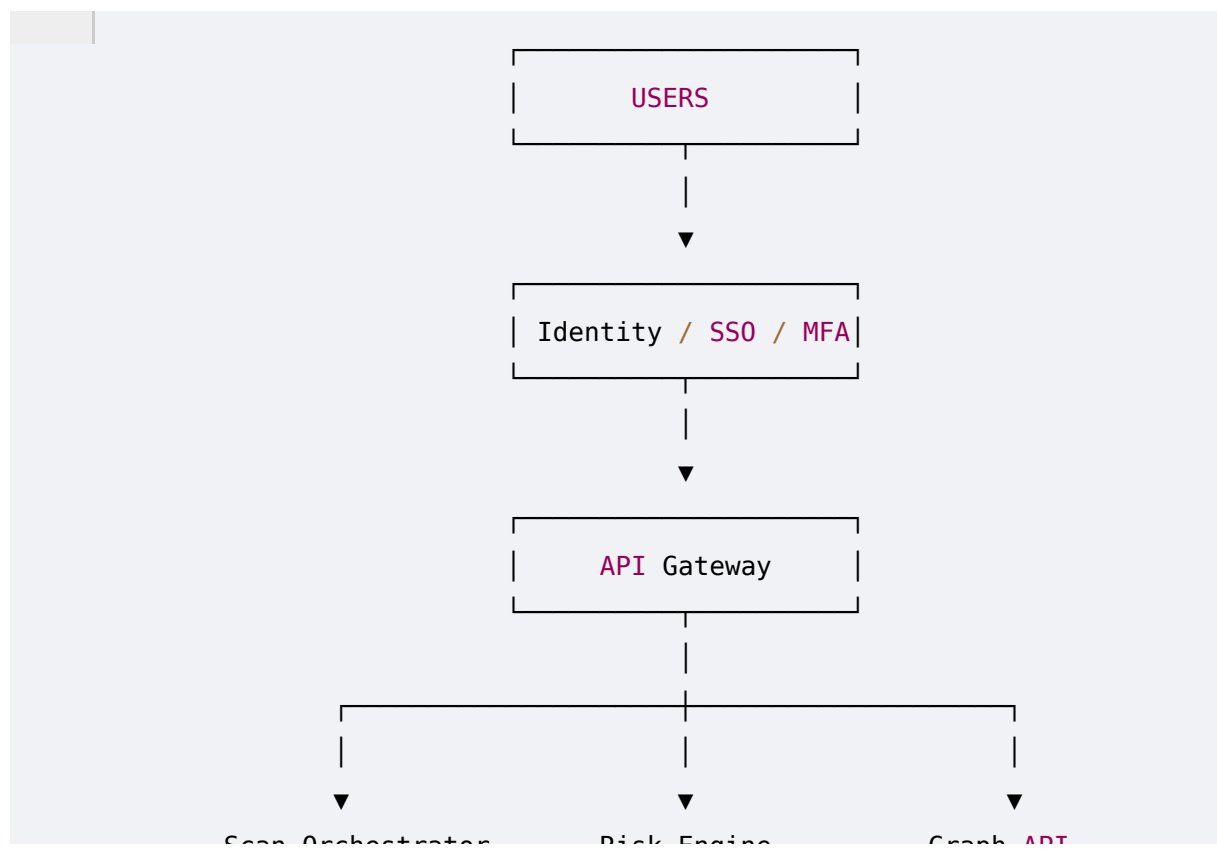
User
↓
Question
↓
Authorization
↓
Context Retrieval
↓
Evidence Filtering
↓
AI Gateway
↓
LLM
↓
Validation
↓
Answer

134. Migration Data Flow

Asset
↓
Risk
↓
Target Registry
↓
Compatibility
↓
Dependencies



135. The Complete Technical Architecture



136. The Final Product Stack

A practical stack could therefore be:

Layer	Technology
Frontend	Next.js + TypeScript

UI	Tailwind
Charts	ECharts
Graph UI	React Flow
Backend	FastAPI
Language	Python
Validation	Pydantic
ORM	SQLAlche my
DB	PostgreS QL
Graph	Neo4j
Queue	Redis + Celery
Storage	MinIO
Parsing	Tree-sitter
Binary	LIEF
Container	Syft/Trivy/ custom
Certificate s	OpenSSL / Python cryptogra phy

AI	AI Gateway + configurable LLM
Auth	OIDC/OAuth2
Deployment	Docker/Kubernetes
CI/CD	GitHub Actions/GitLab CI
Observability	OpenTelemetry

This is a recommended architecture, not a mandatory stack.

137. What NOT to Build Initially

Avoid:

- ✗ 20 microservices
- ✗ Fully autonomous AI agent
- ✗ Automated production migration
- ✗ Complex blockchain infrastructure
- ✗ Custom cryptographic algorithms
- ✗ Custom authentication protocol
- ✗ Huge ML training pipeline
- ✗ Fully dynamic runtime instrumentation

These consume time without improving the SIH prototype enough.

138. What You Absolutely Need

- ✓ Real scanner
- ✓ Real **CBOM**
- ✓ Real evidence
- ✓ Real risk calculation
- ✓ Real dependency graph
- ✓ Real **AI** grounding
- ✓ Real recommendation logic
- ✓ Working migration plan
- ✓ Working rescan
- ✓ Working verification

139. The "Minimum Impressive Product"

If time becomes extremely constrained:

1. Upload Git repository
2. Scan Python/**JS**/Java
3. Detect **RSA/ECDSA/AES/SHA/TLS**
4. Generate **CBOM**
5. Calculate quantum risk
6. Visualize dependency graph
7. Ask **AI** questions
8. Generate **PQC** migration recommendation
9. Generate migration plan
10. Rescan and verify

If those ten work beautifully, you have a serious prototype.

140. Architecture Maturity

Prototype

Docker Compose
Postgres
Neo4j
FastAPI
Next.js
Python scanners
One **AI** provider

Pilot

- Kubernetes
- SSO
- Queue
- Object storage
- Multiple workers
- Observability

Enterprise

- Private deployment
- HSM/KMS
- SIEM
- Air-gapped mode
- Private AI
- Multi-tenant controls
- High availability

141. Implementation Rule

Build in this order:

- FUNCTIONALITY
- ↓
- CORRECTNESS
- ↓
- SECURITY
- ↓
- PERFORMANCE
- ↓
- SCALABILITY
- ↓
- POLISH

Do not reverse this.

A beautiful dashboard on top of an inaccurate scanner is worthless.

142. Phase 21 Final Architecture Principle

ECDAT should fundamentally be:

**A deterministic cryptographic intelligence engine
with an AI-powered analyst interface.**

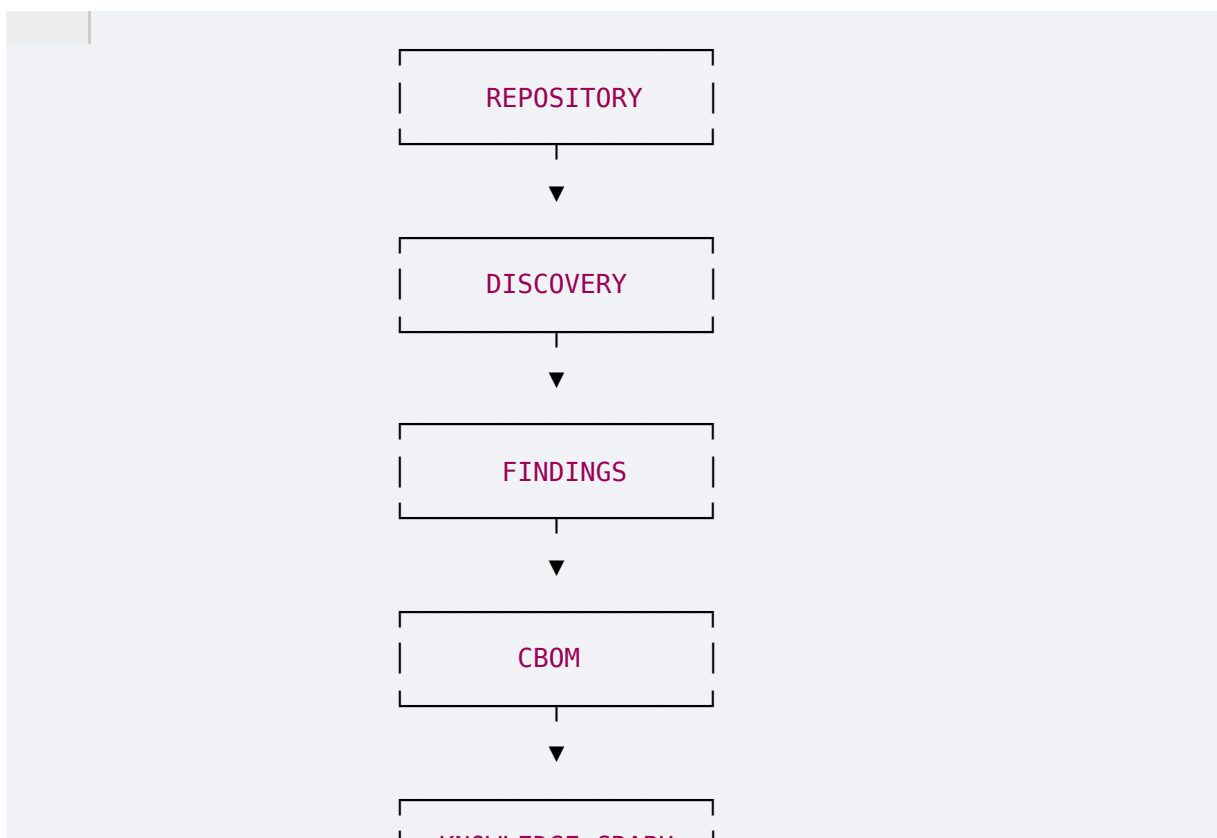
Not:

**An AI application that happens to scan
cryptography.**

That distinction should influence every architectural decision.

143. Phase 21 Final Takeaway

The system now has a clear engineering path:



This is the architecture your team can actually turn into code.

**PHASE 22 — FINAL PHASE: PRODUCTIZATION, SIH
STRATEGY, IMPLEMENTATION ROADMAP, PITCH,
JUDGE QUESTIONS & "HOW TO WIN"**

The final phase will bring the entire 21-phase analysis together.

It will cover:

- Complete ECDAT product definition
- What to build vs what not to build
- 24-hour / 48-hour / 1-week / 1-month execution plans
- Team allocation
- Engineering milestones
- Feature prioritization
- MVP definition
- Prototype definition
- Production roadmap
- Demo architecture
- SIH presentation structure
- Problem statement interpretation
- Innovation claims
- Differentiation
- Judge psychology
- Expected judge questions
- Technical questions
- Cybersecurity questions
- PQC questions
- AI questions
- Architecture questions
- Scalability questions
- Accuracy questions
- "Why not existing tools?"
- "Why AI?"
- "Why graph?"
- "Why CBOM?"
- "Why PQC?"
- "How do you validate recommendations?"
- "Can this work offline?"
- "How do you protect source code?"
- "How do you prevent hallucinations?"

- "How do you handle false positives?"
- "How do you handle binaries?"
- "How do you handle proprietary systems?"
- "What happens after discovery?"
- "What is your actual innovation?"
- Full 5-minute pitch
- Full 10-minute pitch
- Demo script
- Architecture explanation
- Backup demo
- Final feature matrix
- Final technical checklist
- Submission checklist
- Future roadmap
- Commercialization potential
- National-security relevance
- Enterprise adoption story
- Final one-line positioning

And most importantly, Phase 22 will answer:

If six students had to actually build ECDAT for SIH, what exactly should they build, in what order, and what should they show the judges?

That will be the final synthesis of the entire ECDAT architecture.

Phase 21 complete.